

AURIX Backend System Architecture

The AURIX platform functions as an autonomous, agentic unified risk intelligence ecosystem. It integrates intelligent scanning with autonomous patching to provide on-demand security validation for code builds.

1. High-Level Architecture

The backend serves as the central API Gateway and traffic controller, built on Node.js + Express.js. It relies on two primary cloud infrastructures:

- **Supabase (PostgreSQL + Auth + Storage + Vector):** Functions as the primary database, identity provider, large file vault, and vector search engine.
- **Upstash Redis:** Acts as a high-speed message queue and rate-limiter, buffering requests to manage heavy loads.

Structural Flow

1. **Client (Web Dashboard / VS Code):** Sends API requests to the Node.js API Gateway.
2. **Authentication:** Gateway validates requests via Supabase Auth Vault.
3. **Data Storage:** Metadata is stored in Supabase PostgreSQL; zip payloads are stored in Supabase Object Storage.
4. **Processing:** Jobs are pushed to the Upstash Redis Queue, where the AI Worker (Divyansh's component) pulls them for processing.
5. **Resolution:** The AI Worker pushes results back to the Gateway via an Internal Webhook.

2. Core Service Workflows

- **GitHub URL Ingestion:** Frontend sends a `POST /api/scans/github` request with a JWT. The backend creates a `PENDING` scan record in PostgreSQL and pushes a job payload to the Redis Queue.
- **Zip File Upload:** VS Code extension sends `POST /api/scans/upload` as `multipart/form-data`. The file is streamed to Supabase Storage, and the storage path is pushed to the Redis Queue.

- **AI Webhook Resolution:** Once the AI Worker completes a scan, it calls `POST /api/internal/webhook/scan-complete` with a secure token. The backend updates the scan status to `COMPLETED` and writes findings to the `verified_vulnerabilities` table.
- **RAG Threat Intel Search:** The AI Worker hits `POST /api/internal/threat-intel/search` to perform a similarity search against the Supabase `pgvector` extension for contextual threat data.

3. API Endpoints Map

Endpoints for Bhumika (Frontend) and Divyanshi (VS Code) require a Supabase JWT.

Method	Endpoint	Description
GET ▾	/health	Check API status
POST ▾	/api/scans/github	Queue GitHub repo for scanning
POST ▾	/api/scans/upload	Upload .zip for scanning
GET ▾	/api/scans/{scan_id}	Retrieve scan status and findings

4. Team Responsibilities & Dependencies

Backend & Infrastructure Engineer (Bhavya)

- **Role:** Acts as the "Traffic Cop" and "Vault," managing the API Server, Database, S3 Storage, and Redis Queue.
- **Responsibilities:** Implement API routes, maintain database schema, manage auth via Supabase, implement rate limiting, and execute data retention policies (cleaning up raw storage post-scan).
- **Critical Constraint:** All HTTP requests must be asynchronous. The system must accept the request, push to Redis, save to DB, and return `202 Accepted` immediately.

Web Frontend Developer (Bhumika)

- **Role:** Builds the "Face" and "Door" of the platform (Dashboard, Auth UI).
- **Responsibilities:** Develop authentication screens, manage GitHub OAuth flow, implement the polling mechanism for scan status (`GET /api/scans/{scan_id}`), and visualize AI findings in a Kanban board.

Core Engine Architect (Divyansh)

- **Role:** Builds the "Muscle and Brain" (Triage, LangGraph Orchestration, and Sandbox).
- **Responsibilities:** Manage the worker node, execute the "Semantic Slicer," run the Red/Blue agent wargaming loop, and maintain the ephemeral Docker sandbox.
- **Critical Dependency:** Must agree on the final JSON format of `verified_vulnerabilities` with the Backend Engineer before finalizing implementation.